4. Binary Search with Big-O Analysis

Aim: To search for an element in a sorted array using Binary Search and analyze its time complexity.

Algorithm

1. Start the program.
2. Read the size and sorted elements of the array.
3. Read the element to be searched.
4. Set low = 0 and high = n - 1.
5. Calculate the middle position using mid = (low + high) / 2.
6. If the middle element equals the search element, display its position.
7. If the search element is greater, search the right half; otherwise, search the left half.
8. Repeat until the element is found or low > high.
9. If the element is not found, display the appropriate message.
10. Stop the program.

C

```
#include <stdio.h>

int main() {
    int a[100], n, i, key;
    int low, high, mid, found = 0;
    printf("Enter the size of array: ");
    scanf("%d", &n);
    printf("Enter sorted array elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
    printf("Enter element to search: ");
    scanf("%d", &key);
    low = 0;
    high = n - 1;
    while (low <= high) {
        mid = (low + high) / 2;
        if (a[mid] == key) {
```

```
            printf("Element found at position %d", mid + 1);
            found = 1;
            break;
        } else if (a[mid] < key) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    if (found == 0) {
        printf("Element not found");
    }
    return 0;
}
```

Input: Enter sorted array: 10 20 30 40 50 60 70

Enter element to search: 50

Output:

```
Enter the size of array: 7
Enter sorted array elements: 10 20 30 40 50 60 70
Enter element to search: 50
Element found at position 5
```

Result: Thus, the C program successfully searches for an element in a sorted array using Binary Search and analyses its time complexity using Big-O notation.